Chapter 17 Exception Handling

17.1 Introduction

- Error
  - Compilation error
  - Runtime error
    - Logic error
    - Exception
- Exception(异常)
  - Unusual but predictable
- Exception handling(异常处理)
  - Removes error-handling code from the program execution's "main line"
  - Robust(健壮) and fault-tolerant(容错)

17.2 Basics of C++ Exception Handling: try, throw, catch(cont.)

- Exception handling:
  - Throw: `throw`
  - Detect: `try`
  - Handle: `catch`

Every `try` block must be followed immediately with at least one `catch` handler.

❖ Define exception class
- Standard class
- User-defined class

❖ Exception parameter

`catch(<type-of-exception> <parameter>)`

Notice: Throw point must before the codes on which errors might occur.

```
try                                  throw point
{
  throw Exception of type A;
  Codes that might be error;
}
catch( Exception_type A & )
{
   Exception_A handling;
```

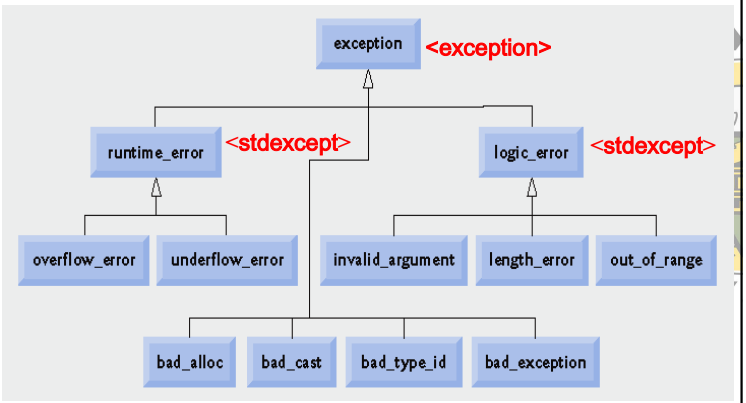

17.2 Basics of C++ Exception Handling: try, throw, catch(cont.)

- Multi-exception handling:

`catch(…)`

```
try
{
  throw Exception_type A ;
  …
  throw Exception_type B ;
}
catch( Exception_type A & )
{
   Exception_A handling;
}
catch( Exception_type B & )
{
   Exception_B handling;
}
```


17.3 Standard Library Exception Hierarchy(cont.)



17.4 Exception and Inheritance

- Inheritance with exception classes
  - New exception classes can be defined to inherit from existing exception classes
  - A `catch` handler for a particular exception class can also catch exceptions of classes derived from that class
  - This allows for polymorphic processing of related errors (多态处理异常)

17.4 Exception and Inheritance (cont.)

- Example:

```
bool flag = true;
try
{
   if(flag)
       throw runtime_error("runtime error");
   else
       throw logic_error("logic error");
}
catch( exception &e )
{
   cout << e.what() << endl; // virtual function what()
}
```


17.5 Example: Handling an Attempt to Divide by 0

```
double quotient( int numerator, int denominator ) {
   if ( denominator == 0 )
      \\ error, should throw an exception
   return <static_cast>(numerator) / denominator;
}
```

```
void main() {
   int number1, number2; double result;
   cout << "Enter 2 integers(end-of-file to end): ";
   while( cin >> number1 >> number2)
   {
      result = quotient( number1, number2 );
      cout<< "The quotient is: " << result << endl;
      cout<<"Enter 2 integers(end-of-file to end): ";
   }
}
```

17.5 Example: Handling an Attempt to Divide by 0

Exception handling -- Step 1: Throw exception

```
double quotient( int numerator, int denominator ) {
   if ( denominator == 0 )
      throw DivideByZeroException(
   return <static_cast>(numerator)
}
```

If the thrown operand is an object, it is called exception object.

```
#include <stdexcept>
using std::runtime_error;

class DivideByZeroException : public runtime_error
{
public:
   DivideByZeroException()
      : runtime_error( "attempted to divide by zero" )
   {}
};
```

Tell exception type

17.5 Example: Handling an Attempt to Divide by 0

Exception handling -- Step 2: Detect exception(try)

```
int main(){
   int number1, number2;
   double result;
   cout<<"Enter 2 integers(end-of-file to end): ";
   while(cin>>number1>>number2)
   {
      try
      {
         result = quotient( number1, number2 );
         cout<< "The quotient is: " << result << endl;
      }
      cout<<"Enter 2 integers(end-of-file to end): ";
   }
}
```

A try block should contain:
① Statements that may cause exceptions

② Statements that should be skipped in case of an exception

17.5 Example: Handling an Attempt to Divide by 0

Exception handling -- Step 3: Handle exception(catch)

```
int main(){
   …
   while(cin>>number1>>number2)
   {
      try
      {
         result = quotient( number1, number2 );
         cout<< "The quotient is: "<< result<< endl;
      }
      catch( DivideByZeroException
                              &divideByZeroException )
      { cout << "Exception occurred: "
             << divideByZeroException.what() << endl;
      }
      cout<<"Enter 2 integ
   }
}
```

throw DivideByZeroException();

Return the error information by virtual function what() defined in base-class exception

17.5 Example: Handling an Attempt to Divide by 0

Results:

```
Enter two integers (end-of-file to end): 100 7
The quotient is: 14.2857

Enter two integers (end-of-file to end): 100 0
Exception occurred: attempted to divide by zero

Enter two integers (end-of-file to end): ^z
```

17.6 Rethrowing an Exception

- Rethrowing an exception

  `throw;`

  – Use when a `catch` handler cannot or can only partially process(部分处理) an exception

  – Next enclosing `try` block attempts to match the exception with one of its `catch` handlers


```
1  // Fig. 17.3: Fig17_03.cpp
2  // Demonstrating exception rethrowing.
3  #include <iostream>
4  using std::cout;
5  using std::endl;
6
7  #include <exception>
8  using std::exception;
9
10 // throw, catch and rethrow exception
11 void throwException()
12 {
13    // throw exception and catch it immediately
14    try
15    {
16       cout << "  Function throwException throws an exception\n";
17       throw exception(); // generate exception
18    } // end try
19    catch ( exception & ) // handle exception
20    {
21       cout << "  Exception handled in function throwException"
22          << "\n  Function throwException rethrows exception";
23       throw; // rethrow exception for further processing
24    } // end catch
25
26    cout << "This also should not print\n";
27 } // end function throwException
```

Rethrow exception()

Also skipped


```
28
29 int main()
30 {
31    // throw exception
32    try
33    {
34       cout << "\nmain invokes function throwException\n";
35       throwException();
36       cout << "This should not print\n";
37    } // end try
38    catch ( exception & ) // handle exception
39    {
40       cout << "\n\nException handled in main\n";
41    } // end catch
42
43    cout << "Program control continues after catch in main\n";
44    return 0;
45 } // end main
```

Catch rethrown exception

Skipped

```
main invokes function throwException
   Function throwException throws an exception
   Exception handled in function throwException
   Function throwException rethrows exception

Exception handled in main
Program control continues after catch in main
```

嵌套异常处理过程

```
main(){
  try{
    cout << ……
    throwExeption()
    cout << …… // skip
  }
  catch(exception &){
    cout << ……
  }
  cout << ……
}
```

```
throwException(){
  try{
    cout << ……
    throw exception();
  }
  catch(exception &){
    cout << ……
    throw;
  }
  cout << ……  //skip
}
```

①②③④

17.7 Exception Specifications

- Exception specifications (`throw` lists)

  – Enumerates a list of exceptions that a function can throw

```
int someFunction(double value)
throw( ExceptionA, ExceptionB, ExceptionC )
{
   … …
}
```

  – A function can throw only exceptions of types in its specification or types derived from those types

17.7 Exception Specifications(cont.)

- Exception specifications (cont.)

  – No exception specification indicates the function can throw any exception

    - e.g.: `void func(){…}`

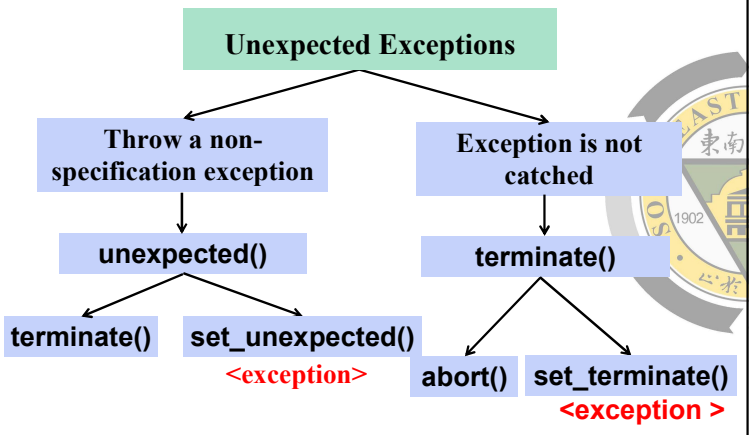
  – An empty exception specification, `throw()`, indicates the function can not throw any exception

    - e.g.: `void func() throw() {…}`

  – If a function throws a non-specification exception, function `unexpected()` is called

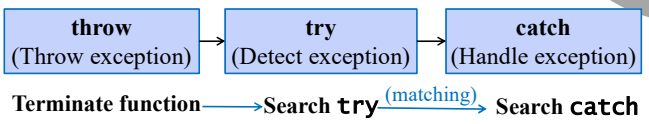
    - Normally call the function `terminates()` to end the program

16.8 Processing Unexpected Exceptions(cont.)



17.9 Stack Unwinding(cont.)

- Stack unwinding(栈展开)
  - Occurs when a thrown exception is not caught in a particular scope.
  - Attempts are made to `catch` the exception in outer `try`…`catch` blocks
  - Termination model
    - When an exception is thrown, the function in which the exception is not caught terminates, then search for a `try` block and a matching `catch` handler in outer scope



Terminate function → Search `try` (matching) → Search `catch`


```
34 int main()
35 {
37    try
38    {
39       cout << "function1 is called inside main" << endl;
40 ①    function1();
41    }
42    catch ( runtime_error &error )
43    {
44       cout << "Exception occurred: " << error.what() << endl;
45       cout << "Exception handled in main" << endl;
46    }
47
48    return 0;
49 }
```



```
1  // Fig. 17.4: Fig17_04.cpp
2  // Demonstrating stack unwinding.
3
11 void function3() throw ( runtime_error )
12 {
13    cout << "In function 3" << endl;
16 ④ throw runtime_error( "runtime_error in function3" );
17 }
18
20 void function2() throw ( runtime_error )
21 {
22    cout << "function3 is called inside function2" << endl;
23 ⑤ function3();
24 }
25
27 void function1() throw ( runtime_error )
28 {
29    cout << "function2 is called inside function1" << endl;
30 ⑥ function2();
31 }
```

throw is not enclosed in try block (terminate function3())

throw is not enclosed in try block (terminate function2())

throw is not enclosed in try block (terminate function1())


```
34 int main()
35 {
37    try
38    {
39       cout << "function1 is called inside main" << endl;
40 ⑦    function1();
41    }
42 ⑧ catch ( runtime_error &error )
43    {
44       cout << "Exception occurred: " << error.what() << endl;
45 ⑨    cout << "Exception handled in main" << endl;
46    }
47
48 ⑩ return 0;
49 }
```

throw is enclosed in try block

catch handler: matched

execute catch handler

```
function1 is called inside main
function2 is called inside function1
function3 is called inside function2
In function 3
Exception occurred: runtime_error in function3
Exception handled in main
```

resume the first statement after the last catch handler

17.10 Constructors, Destructors and Exception Handling

- Exceptions and constructors
  - Object is partially constructed
  - Destructors are called for all automatic objects in the terminated `try` block when an exception is thrown
    - If an object has member objects, destructors will be executed for the member objects that have been constructed prior to the occurrence of the exception.
    - If an array of objects has partially constructed when an exception occurs, only the destructors for the constructed objects in the array will be called.

17.10 Constructors, Destructors and Exception Handling(cont.)

- Exceptions and destructors
  - If a destructor invoked by stack unwinding throws an exception, function `terminate` is called
  - Usually, destructors donnot throw exceptions !

17.11 Processing new Failures

- new failures
  - Some compilers throw a bad_alloc exception
    - Compliant（服从） to the C++ standard specification
  - Some compilers return 0
    - C++ standard-compliant compilers also have a version of new that returns 0
  - Some compilers throw bad_alloc if <new> is included


```
1  // Fig. 16.5: Fig16_05.cpp
2  // Demonstrating pre-standard new returning 0 when memory
3  // is not allocated.
4  #include <iostream>
5  using std::cerr;
6  using std::cout;
7
8  int main()
9  {
10    double *ptr[ 50 ];
11
12    // allocate memory for ptr
13    for ( int i = 0; i < 50; i++ )
14    {
15       ptr[ i ] = new double[ 50000000 ];
16
17       if ( ptr[ i ] == 0 ) // did new fail to allocate memory
18       {
19          cerr << "Memory allocation failed for ptr[ " << i << " ]\n";
20          break;
21       } // end if
22       else // successful memory allocation
23          cout << "Allocated 50000000 doubles in ptr[ " << i << " ]\n";
24    } // end for
25
26    return 0;
27 } // end main
```

Allocate 50000000 double values

new will have returned 0 if the memory allocation operation failed

```
Allocated 50000000 doubles in ptr[ 0 ]
Allocated 50000000 doubles in ptr[ 1 ]
Allocated 50000000 doubles in ptr[ 2 ]
Memory allocation failed for ptr[ 3 ]
```



```
1  // Fig. 16.6: Fig16_06.cpp
2  // Demonstrating standard new throwing bad_alloc when memory
3  // cannot be allocated.
4  #include <iostream>
5  using std::cerr;
6  using std::cout;
7  using std::endl;
8
9  #include <new> // standard operator new
10 using std::bad_alloc;
11
12 int main()
13 {
14    double *ptr[ 50 ];
15
16    // allocate memory for ptr
17    try
18    {
19       // allocate memory for ptr[ i ]; new throws bad_alloc on failure
20       for ( int i = 0; i < 50; i++ )
21       {
22          ptr[ i ] = new double[ 50000000 ]; // may throw exception
23          cout << "Allocated 50000000 doubles in ptr[ " << i << " ]\n";
24       } // end for
25    } // end try
```

Allocate 50000000 double values


```
26
27    // handle bad_alloc exception
28    catch ( bad_alloc &memoryAllocationException )
29    {
30       cerr << "Exception occurred: "
31          << memoryAllocationException.what() << endl;
32    } // end catch
33
34    return 0;
35 } // end main
```

new throws a **bad_alloc** exception if the memory allocation operation failed

```
Allocated 50000000 doubles in ptr[ 0 ]
Allocated 50000000 doubles in ptr[ 1 ]
Allocated 50000000 doubles in ptr[ 2 ]
Exception occurred: bad allocation
```

17.11 Processing new Failures(cont.)

To make programs more robust, use the version of new that throws bad_alloc exceptions on failure.

17.11 Processing new Failures(cont.)

- new failures (cont.)
  - set_new_handler(…) (included in <new>)
    - Registers a function to handle new failures
      - The registered function is called by new when a memory allocation operation fails
    - Takes as argument a pointer to a function that takes no arguments and returns void
    - The new-handler function should:
      - Make more memory available and let new try again,
      - Throw a bad_alloc exception or
      - Call abort or exit to terminate the program


```
1  // Fig. 16.7: Fig16_07.cpp
2  // Demonstrating set_new_handler.
3  #include <iostream>
4  using std::cerr;
5  using std::cout;
6
7  #include <new> // standard operator new and set_new_handler
8  using std::set_new_handler;
9
10 #include <cstdlib> // abort function prototype
11 using std::abort;
12
13 // handle memory allocation failure
14 void customNewHandler()
15 {
16    cerr << "customNewHandler was called";
17    abort();
18 } // end function customNewHandler
19
20 // using set_new_handler to handle failed memory allocation
21 int main()
22 {
23    double *ptr[ 50 ];
24
25    // specify that customNewHandler should be called on
26    // memory allocation failure
27    set_new_handler( customNewHandler );
```

Create a user-defined new-handler function customNewHandler

Register customNewHandler with set_new_handler


```
28
29    // allocate memory for ptr[ i ]; customNewHandler will be
30    // called on failed memory allocation
31    for ( int i = 0; i < 50; i++ )
32    {
33       ptr[ i ] = new double[ 50000000 ]; // may throw exception
34       cout << "Allocated 50000000 doubles in ptr[ " << i << " ]\n";
35    } // end for
36
37    return 0;
38 }  // end main
```

Allocate 50000000 double values

```
Allocated 50000000 doubles in ptr[ 0 ]
Allocated 50000000 doubles in ptr[ 1 ]
Allocated 50000000 doubles in ptr[ 2 ]
customNewHandler was called
```